FOUNDATIONS & BIG-O ANALYSIS

FAANG Interview Orientation — Mentor Guide

🎬 THE REAL-WORLD STORY

Imagine transferring a 100 GB file across town. Wi-Fi scales linearly with file size (100 GB = 200 min) \rightarrow **O(N)**. Driving a USB drive takes fixed time regardless of size (100 GB = 30 min) \rightarrow **O(1)**.

💡 THE BIG-O AHA MOMENT

Big-O notation measures how runtime or memory scales as input size N grows to infinity, ignoring hardware speed differences!

🧐 WHY NOT MEASURE SECONDS?

- A fast CPU runs bad code quickly on tiny inputs.
- OS background processes alter millisecond execution timings.
- Big-O gives a universal mathematical upper bound guaranteed on all FAANG evaluation servers!

📈 BIG-O GROWTH HIERARCHY

Big-O	Name	Max N for 1 Sec (10^8 ops)
O(1)	Constant	Unlimited
O(log N)	Logarithmic	10^18 (Binary Search)
O(N)	Linear	10^8 (Single Pass)
O(N log N)	Linearithmic	10^6 (Sorting)
O(N^2)	Quadratic	10^4 (Nested Loops)
O(2^N) / O(N!)	Exponential	20 / 11 (Recursion)

📐 THE 3 BIG-O REDUCTION RULES

Rule 1 — Drop Constants: O(2N + 5) \rightarrow **O(N)**. We care about growth rate, not constant multipliers.

Rule 2 — Drop Non-Dominant Terms: O(N^2 + 100N + 500) \rightarrow **O(N^2)**. N^2 dominates N as N grows.

Rule 3 — Multi-Variable Rules: Array A size N, Array B size M \rightarrow **O(N + M)** for sequential loops, **O(N · M)** for nested loops!

⚡ 5-STEP FAANG PROBLEM SOLVING FRAMEWORK

1. **Clarify Constraints:** Ask N limits (N <= 10^5 \implies O(N log N) target).
2. **State Brute Force:** Mention naive solution & complexity first.
3. **Identify Bottleneck:** Spot repeated work \rightarrow HashMap/Binary Search.
4. **Optimize & Code:** Write clean, modular Java standard code.
5. **Dry Run Test Cases:** Step line-by-line with small/edge inputs!

📊 DEDUCING TARGET COMPLEXITY DIRECTLY FROM INPUT CONSTRAINTS (N)

Input Size (N)	Target Time Complexity	Typical Algorithm Patterns
N <= 10..12	O(N!) or O(2^N · N)	Permutations, N-Queens, Backtracking Bitmask
N <= 20..25	O(2^N)	Subsets, Combination Sum, Meet-in-the-Middle
N <= 500	O(N^3)	Floyd-Warshall, Matrix Chain Multiplication DP
N <= 2,000	O(N^2)	Nested Loops, 2D Grid DP, All-Pairs Check
N <= 10^5..10^6	O(N log N) or O(N)	Sorting, Binary Search, Two Pointers, HashMap, Sliding Window
N <= 10^9..10^18	O(log N) or O(1)	Binary Search on Answer, Bit Manipulation, Math Formulas

JAVA TOOLKIT & FAANG DRY RUN PROTOCOL

Auxiliary Memory, Pitfalls & Verification Protocol

💡 AUXILIARY SPACE VS CALL STACK

- **Input Space:** Memory used to store input data (e.g. array of size N).
- **Auxiliary Space:** Extra memory created by your code (DS + call stack).
- **Recursion Call Stack:** `factorial(3)` calls `factorial(2)` \rightarrow Max stack depth 3 \implies **O(N) Auxiliary Space!**

🛠 ESSENTIAL JAVA DATA STRUCTURE COMPLEXITIES

Data Structure	Operation	Time Complexity
`ArrayList`	`add()`, `get(i)`, `remove(last)`	O(1) amortized
`HashMap`	`put()`, `get()`, `containsKey()`	O(1) average, O(N) worst
`HashSet`	`add()`, `contains()`, `remove()`	O(1) average
`ArrayDeque`	`push()`, `pop()`, `offer()`, `poll()`	O(1) strictly faster than Stack
`PriorityQueue`	`offer()`, `poll()`, `peek()`	O(log N) offer/poll, O(1) peek

⚡ JAVA INTERVIEW POWER SNIPPETS

```
// 1. Frequency Map Shortcut:
map.put(key, map.getOrDefault(key, 0) + 1);

// 2. Sort Array in Reverse Order:
Arrays.sort(arr, (a, b) → Integer.compare(b,
a));

// 3. Min Heap & Max Heap Declarations:
PriorityQueue<Integer> minHeap = new PriorityQueu
e<>();
PriorityQueue<Integer> maxHeap = new PriorityQueu
e<>((a, b) → b - a);

// 4. String Array to Char Manipulation:
char[] chars = str.toCharArray();
String sortedStr = new String(chars);
```

💼 COMMON JAVA INTERVIEW BUGS & FIXES

1. Binary Search Mid Overflow:
`int mid = (left + right) / 2;` \rightarrow ❌ Overflows if `left + right > 2^31 - 1`!
`int mid = left + (right - left) / 2;` \rightarrow ✅ Safe formula!

2. String Immutability Loop Overhead:
`String s += "a";` in loop \rightarrow ❌ Recreates string in O(N^2) total time!
`StringBuilder sb = new StringBuilder(); sb.append('a');` \rightarrow ✅ O(1) amortized!

🔍 THE 4-STEP FAANG DRY RUN METHOD

1. **Pick a Small Concrete Input:** e.g. `nums = [2, 7, 11]`, `target = 9`.
2. **Create a Variable State Table:** Track loop counters, pointers, & variables.
3. **Step Line-by-Line:** Execute code manually without skipping steps!
4. **Test Edge Cases:** Check empty inputs, 1-element arrays, & duplicates.

📊 SAMPLE VARIABLE STATE TRACE TABLE

Line	i	nums[i]	complement	map state	Action
1	0	2	7	{}	`map.put(2, 0)`
2	1	7	2	{2: 0}	Match Found! Return `[0, 1]`

✅ ORIENTATION MASTERY CHECKLIST

☐ Explain Big-O using real-world analogies (Wi-Fi vs Car)
☐ Deduce target complexity directly from input size N
☐ State 3 Big-O reduction rules without hesitation
☐ Use safe mid formula `left + (right - left) / 2`
☐ Differentiate Input Space vs Auxiliary Call Stack Space

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — State Brute Force First
Always state the obvious brute force solution and its time/space complexity before jumping into optimization! It shows structured thinking.

☐ Perform 4-step line-by-line variable trace dry runs

Rule 2 — Dry Run Saves You from Rejection

90% of interview bugs (off-by-one errors, null pointers, unhandled edge cases) are caught during your dry run before running tests!